

---

# LAMBDA DISCREPANCY IN REINFORCEMENT LEARNING

---

A PREPRINT

Marwin Prenner

June 2025

## ABSTRACT

### Abstract

Teilweise beobachtbare Umgebungen gehören zu den grundlegendsten Herausforderungen im Reinforcement Learning (RL): Wichtige Informationen über den Zustand sind nicht direkt zugänglich, sondern müssen aus Beobachtungsfolgen erschlossen werden. Rekurrente neuronale Netze wie LSTMs bieten eine mögliche Lösung – doch bislang fehlt eine systematische Methode, um zu bestimmen, *wann* ein Gedächtnis tatsächlich erforderlich ist und *wie* dessen Lernprozess gezielt unterstützt werden kann.

Diese Arbeit untersucht die *Lambda Discrepancy (LD)* – eine neuartige, theoretisch motivierte Metrik zur Einschätzung partieller Beobachtbarkeit. LD basiert auf dem Vergleich von Wertschätzungen aus dem  $TD(\lambda)$ -Lernen mit unterschiedlichen Zeithorizonten und dient als auxiliary loss, um in rekurrenten Architekturen die Bildung nützlicher interner Repräsentationen zu fördern.

Zur experimentellen Evaluation wurde eine PPO-basierte Trainingspipeline in einer angepassten Version von *Pokémon Red* realisiert. Diese Umgebung zeichnet sich durch visuelle Komplexität, teilbeobachtbare Dynamik und explorationsbasiertes Gameplay aus. Drei eigene Gym-kompatible Varianten ermöglichen eine differenzierte Untersuchung rekurrenter Modelle mit und ohne LD. Zum Zeitpunkt der Einreichung war das Training noch nicht abgeschlossen.

**Keywords** Reinforcement Learning · Partial Observability · Lambda Discrepancy · Auxiliary Loss · LSTM

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Einleitung</b>                                       | <b>3</b> |
| <b>2</b> | <b>Related Work &amp; Methodological Foundations</b>    | <b>3</b> |
| 2.1      | Reinforcement Learning und MDPs . . . . .               | 3        |
| 2.2      | Gedächtnisarchitekturen in POMDPs . . . . .             | 3        |
| 2.3      | Lambda Discrepancy als Trainingssignal . . . . .        | 4        |
| 2.4      | Alternative Strategien zur POMDP-Bewältigung . . . . .  | 4        |
| 2.5      | Praktische Anwendungen rekurrenter RL-Modelle . . . . . | 4        |
| <b>3</b> | <b>Methodik</b>                                         | <b>4</b> |
| 3.1      | Untersuchungsdesign . . . . .                           | 4        |
| 3.2      | Trainingsumgebung . . . . .                             | 5        |
| 3.3      | Architekturen und Loss-Funktionen . . . . .             | 5        |
| 3.4      | Evaluationskriterien . . . . .                          | 5        |

---

|            |                                                                  |          |
|------------|------------------------------------------------------------------|----------|
| 3.5        | Reproduzierbarkeit . . . . .                                     | 5        |
| <b>4</b>   | <b>Diskussion und Ausblick</b>                                   | <b>6</b> |
| 4.1        | Methodische Reflexion . . . . .                                  | 6        |
| 4.2        | Theoretisch erwartbare Effekte . . . . .                         | 6        |
| 4.3        | Ausblick . . . . .                                               | 6        |
| <b>A</b>   | <b>Appendix</b>                                                  | <b>6</b> |
| <b>A.1</b> | <b>Markov Decision Processes</b>                                 | <b>6</b> |
| <b>A.3</b> | <b>TD(<math>\lambda</math>) und <math>\lambda</math>-Returns</b> | <b>8</b> |
| <b>A.4</b> | <b>Lambda Discrepancy: Formale Grundlage</b>                     | <b>8</b> |
| <b>A.5</b> | <b>Gedächtnislernen bei POMDPs</b>                               | <b>9</b> |

# 1 Einleitung

Reinforcement Learning (RL) hat sich in den letzten Jahren als leistungsfähiger Ansatz für sequentielle Entscheidungsprozesse etabliert Sutton and Barto [2018]. In vielen realitätsnahen Szenarien ist jedoch der Zustand der Umgebung nicht vollständig beobachtbar, wodurch klassische RL-Algorithmen an ihre Grenzen stoßen. Solche Situationen lassen sich als *Partially Observable Markov Decision Processes* (POMDPs) modellieren, in denen Agenten lernen müssen, relevante Informationen aus unvollständigen Beobachtungen über die Zeit hinweg zu rekonstruieren.

Während rekurrente neuronale Netzwerke, insbesondere LSTMs, in der Vergangenheit erfolgreich zur Bewältigung solcher Probleme eingesetzt wurden, bleibt die Frage offen, wie man systematisch erkennen kann, *wann* Gedächtnisstrukturen tatsächlich notwendig sind – und wie sich deren Einsatz gezielt optimieren lässt.

Ein vielversprechender neuer Ansatz ist die sogenannte *Lambda Discrepancy* (LD), eine Metrik zur Quantifizierung partieller Beobachtbarkeit, wie sie von Allen et al. eingeführt wurde Allen et al. [2024]. Sie basiert auf dem Vergleich von zwei  $TD(\lambda)$ -Wertfunktionen mit unterschiedlichen Zeithorizonten Sutton and Barto [2018] und dient als Diagnoseinstrument für nicht-markovsche Entscheidungsgrundlagen. Wird LD als auxiliary loss in rekurrente Architekturen eingebunden, kann sie die Ausbildung interner Gedächtnisrepräsentationen gezielt fördern.

In dieser Arbeit wird LD erstmals in einer visuell komplexen POMDP-Umgebung – dem Gameboy-Spiel *Pokémon Red* – systematisch evaluiert. Auf Basis einer rekurrenten PPO-Trainingspipeline werden vier Agentenvarianten implementiert: klassische Baselines, rekurrente Modelle mit und ohne LD.

Ziel ist es, in einem kontrollierten Experimentier-Setup zu untersuchen, ob und in welchem Ausmaß die Kombination von LSTM und Lambda Discrepancy zu einer stabileren und leistungsfähigeren Policy führt als klassische RL-Ansätze.

**Forschungsfrage:** Führt die Kombination rekurrenter Netzwerke (LSTM) mit Lambda Discrepancy zu einer stabileren und leistungsfähigeren Policy in komplexen POMDP-Umgebungen im Vergleich zu klassischen PPO-Ansätzen?

Zum Zeitpunkt der Abgabe war das Training der Agenten noch nicht vollständig abgeschlossen. Der Fokus dieses Papers liegt daher auf dem methodischen Aufbau, der Trainingsarchitektur und der experimentellen Infrastruktur – alle Komponenten sind vollständig vorbereitet, um künftige Auswertungen und Replikationen zu ermöglichen.

## 2 Related Work & Methodological Foundations

### 2.1 Reinforcement Learning und MDPs

Reinforcement Learning (RL) ist ein Lernparadigma, bei dem ein Agent durch Interaktion mit einer Umgebung eine optimale Handlungsstrategie (Policy) erlernt. In jedem Zeitschritt wählt der Agent eine Aktion auf Basis seiner aktuellen Wahrnehmung, erhält daraufhin eine Belohnung sowie eine neue Beobachtung und passt seine Strategie entsprechend an. Ziel ist es, eine Policy zu finden, die den erwarteten kumulierten Belohnungswert maximiert Sutton and Barto [2018].

In vielen RL-Anwendungen wird das zugrunde liegende Entscheidungsproblem als Markov Decision Process (MDP) modelliert, bei dem angenommen wird, dass die Umwelt vollständig beobachtbar ist. Der Agent kennt zu jedem Zeitpunkt den exakten Zustand, der die Entscheidungsgrundlage bildet. Die Dynamik des Prozesses ist dabei Markovian, d. h. der nächste Zustand hängt nur vom aktuellen Zustand und der gewählten Aktion ab.

Ein *Partially Observable Markov Decision Process* (POMDP) folgt derselben Markov-Dynamik – jedoch hat der Agent keinen direkten Zugang zum zugrunde liegenden Zustand, sondern erhält lediglich Beobachtungen, die diesen nur unvollständig reflektieren. Um dennoch gute Entscheidungen zu treffen, muss der Agent vergangene Beobachtungen und Aktionen berücksichtigen – etwa über ein internes Gedächtnis oder einen Zustandsfilter Kaelbling et al. [1998]. Weitere Details zur formalen Definition von POMDPs sowie deren Bedeutung im Kontext moderner RL-Architekturen finden sich in Anhang A.2.

### 2.2 Gedächtnisarchitekturen in POMDPs

In vollständig beobachtbaren Szenarien genügt es oft, aktuelle Zustandsinformationen zur Entscheidungsfindung heranzuziehen. Bei partiellem Zustandseinblick ist dies nicht ausreichend: Der Agent benötigt ein internes Gedächtnis, um relevante Aspekte vergangener Interaktionen zu speichern und zu berücksichtigen.

Ein bewährter Ansatz zur Integration solcher Gedächtnisstrukturen ist der Einsatz rekurrenter neuronaler Netze (RNNs), die Informationen über Zeit hinweg akkumulieren. Besonders geeignet für RL-Szenarien sind Long Short-Term

Memory Netzwerke (LSTM), da sie durch ihre spezifische Architektur langfristige Abhängigkeiten verarbeiten können Hochreiter and Schmidhuber [1997].

In einem POMDP erweitert ein LSTM-basiertes Modell die Beobachtungshistorie zu einem kompakten latenten Zustand, der als Entscheidungsgrundlage dient. Damit fungiert das LSTM als eine Form der Zustandsschätzung und ermöglicht die Anwendung klassischer RL-Algorithmen auch in teilbeobachtbaren Umgebungen. Die prinzipielle Rolle solcher Gedächtnismodelle in POMDP-Umgebungen wird in Anhang A.5 vertiefend behandelt.

### 2.3 Lambda Discrepancy als Trainingssignal

Ein vielversprechender neuartiger Ansatz zur Einschätzung der Repräsentationsqualität bei partiellem Zustandseinblick ist die *Lambda Discrepancy (LD)*, eingeführt von Allen et al. [2024]. Sie misst, wie stark sich zwei Wertfunktionen unterscheiden, die mit unterschiedlicher zeitlicher Gewichtung  $\lambda$  durch das TD( $\lambda$ )-Verfahren berechnet wurden.

In vollständig beobachtbaren Umgebungen stimmen diese Schätzungen in der Regel gut überein. In POMDPs hingegen können erhebliche Diskrepanzen auftreten, da die Beobachtungen oft nicht alle relevanten Informationen enthalten. Eine hohe LD indiziert somit, dass die aktuelle Repräsentation keine hinreichend markovsche Entscheidungsgrundlage bietet – und dass ein Gedächtnismechanismus erforderlich sein könnte.

Für eine formale Definition und theoretische Begründung der Lambda Discrepancy sei auf Anhang A.4 verwiesen. Die dort dargestellten Eigenschaften liefern die Grundlage für die Integration von LD als differenzierbares Trainingssignal.

### 2.4 Alternative Strategien zur POMDP-Bewältigung

Neben rekurrenten Architekturen mit auxiliary loss wie der Lambda Discrepancy existieren weitere aktuelle Ansätze, um Partial Observability im Reinforcement Learning zu adressieren.

Ein prominenter Zugang sind Belief-State-Encodings durch gezielte Repräsentationslernen. Wang et al. [2023] schlagen vor, latente Zustände in einem kontinuierlichen Raum zu approximieren, der aus Beobachtungssequenzen gelernt wird. Diese Belief-Repräsentationen ermöglichen ein explizites Tracking verborgener Zustände, bleiben jedoch stark von der Qualität des Encoders abhängig.

Ein zweiter Ansatz setzt bei der Unvollständigkeit der Beobachtungen selbst an. Gilmour et al. [2021] schlagen eine dual-policy Architektur vor, bei der neben der klassischen Aktionsauswahl zusätzlich entschieden wird, ob ein Beobachtungsschritt überhaupt durchgeführt werden soll. Dadurch kann der Agent die Informationsaufnahme aktiv steuern, was sich insbesondere in hochfrequenten Spielumgebungen als vorteilhaft erweist.

Ein verwandter, der LD-Methodik ähnlicher Ansatz stammt von Meng et al. [2021]: Sie kombinieren rekurrente Netzwerke wie LSTMs mit einem speziell entwickelten memory embedding, um vergangene Beobachtungen strukturiert zu speichern und abrufbar zu machen. Anders als bei Gu et al. steht hier nicht die Diagnose von Partial Observability im Fokus, sondern das möglichst effektive Lernen einer langzeitfähigen Gedächtnisstruktur.

### 2.5 Praktische Anwendungen rekurrenter RL-Modelle

Gedächtnisbasierte RL-Architekturen finden zunehmend Einsatz in anspruchsvollen Multiagenten- oder High-Dimensional-POMDP-Settings. So demonstrieren Foerster et al. [2016], wie sich mit agentenzentrierter Gedächtnisbildung koordinierte Handlungsstrategien entwickeln lassen. Wie Allen et al. [2024] zeigen, kann die Kombination aus rekurrenten Agenten und Lambda Discrepancy in anspruchsvollen Benchmarks wie *PO-PacMan* oder *RockSample (15,15)* zu signifikant stabileren Policies führen. Die Kombination aus rekursiver Architektur und diagnostischem Zusatzsignal scheint besonders geeignet, wenn relevante Information nicht direkt beobachtbar, sondern über viele Zeitschritte verteilt ist.

## 3 Methodik

### 3.1 Untersuchungsdesign

Ziel dieser Arbeit ist es, die Wirkung eines auxiliary loss auf Basis der Lambda Discrepancy (LD) im Zusammenspiel mit rekurrenten Architekturen systematisch zu untersuchen. Zu diesem Zweck wird ein vergleichendes Trainingssetup mit vier Agentenvarianten eingesetzt, die schrittweise hinsichtlich ihrer Gedächtnisfähigkeit und Repräsentationsnutzung erweitert werden.

- **v1** – PPO mit Frame-Stacking (3 Frames): kontextfreier Agent ohne explizites Gedächtnis.
- **v2** – PPO mit Einzelbild: reduzierteste Form, dient als Vergleichsbasis für spätere Varianten.
- **v3** – Recurrent PPO mit LSTM: Gedächtnisbildung durch rekurrente Repräsentation.
- **v4** – Recurrent PPO mit LSTM + Lambda Discrepancy Loss: Kombination aus Gedächtnismodul und auxiliary loss auf Basis von  $TD(\lambda)$ -Diskrepanzen.

Die zentrale Hypothese lautet, dass die zusätzliche Optimierung über LD insbesondere in teilbeobachtbaren Umgebungen zu stabileren Policies und höherer Gesamteffektivität führt.

### 3.2 Trainingsumgebung

Die Experimente werden in einer angepassten Version von *Pokémon Red* durchgeführt, die über den PyBoy-Emulator und eine Gymnasium-kompatible API angebunden ist. Die Umgebung ist visuell komplex, enthält zufallsabhängige Ereignisse und ist inhärent teilbeobachtbar – was sie für die Analyse von Gedächtnisprozessen besonders geeignet macht.

**Eingabedaten:** Je nach Variante werden einzelne Graustufenbilder (72×80 Pixel) oder ein Frame-Stack mit 3 Bildern als Beobachtung verwendet. Varianten mit Gedächtnis verarbeiten zusätzlich vergangene Aktionen und interne Zustände.

**Emulatorsteuerung:** Alle Episoden starten aus identischen, vorab gespeicherten Emulator-Zuständen. Aktionen werden synchron zum Gameboy-Zyklus alle 24 Frames ausgeführt.

**Hinweis zur Implementierung:** Details zur technischen Anbindung, Frame-Synchronisierung, Snapshot-Verwaltung sowie Emulatorsteuerung sind im zugehörigen GitHub-Repository dokumentiert.<sup>1</sup>

### 3.3 Architekturen und Loss-Funktionen

Alle Agentenvarianten basieren auf Stable Baselines3 und nutzen angepasste Netzwerktopologien. Rekurrente Varianten enthalten ein LSTM-Modul zur Verarbeitung von Sequenzen. Die v4-Variante wird zusätzlich mit einem auxiliary loss auf Basis der Lambda Discrepancy trainiert.

Dieser auxiliary loss berechnet die mittlere quadratische Abweichung zwischen zwei  $TD(\lambda)$ -Schätzungen mit unterschiedlichen Zeitkonstanten ( $\lambda = 0$  und  $\lambda = 1$ ) und ergänzt die PPO-Verlustfunktion:

$$\mathcal{L}_{\text{gesamt}} = \mathcal{L}_{\text{PPO}} + \alpha \cdot \underbrace{\|Q^{\lambda=0} - Q^{\lambda=1}\|_2^2}_{\text{Lambda Discrepancy}}$$

Der Parameter  $\alpha$  steuert dabei den Einfluss des Zusatzterms. Diese Formulierung basiert auf Allen et al. [2024] und wird im Appendix genauer beschrieben.

### 3.4 Evaluationskriterien

Die Evaluierung erfolgt entlang folgender Metriken:

- **Episodischer Reward:** Mittelwert und Varianz über Trainingsepisoden.
- **Lernkurvenstabilität:** Konvergenzverhalten und Sensitivität gegenüber Random Seeds.
- **Spielinterner Fortschritt:** z.B. besiegte Arenaleiter, Team-Level, durchlaufene Gebiete.
- **Explorationsverhalten:** Visualisiert über aggregierte Heatmaps der Spielerbewegungen.

### 3.5 Reproduzierbarkeit

Die Trainingsbedingungen sind vollständig versioniert und dokumentiert. Konfigurationen, Modelle, Umgebungsskripte sowie Logging- und Auswertetools sind im GitHub-Repository verfügbar.<sup>2</sup>

<sup>1</sup><https://github.com/prennerm/PokemonRedExperiments/>

<sup>2</sup><https://github.com/prennerm/PokemonRedExperiments/>

## 4 Diskussion und Ausblick

Auch wenn zur Abgabezeit noch keine vollständigen Trainingsdaten vorliegen, lassen sich auf Grundlage des methodischen Designs sowie bestehender Literatur bereits fundierte Einschätzungen ableiten. Die vorliegende Arbeit schafft eine systematische Vergleichsbasis für den Einsatz der Lambda Discrepancy (LD) als auxiliary loss in rekurrenten RL-Architekturen und ermöglicht so die gezielte Analyse ihres Beitrags zur Bewältigung partieller Beobachtbarkeit.

### 4.1 Methodische Reflexion

Die Integration eines theoretisch motivierten Zusatzsignals (LD) in eine rekurrente PPO-Architektur erlaubt eine differenzierte Untersuchung der Frage, unter welchen Bedingungen Gedächtnisstrukturen und Diagnosemetriken wie LD zusammen wirken. Durch die klare Trennung der Modellvarianten (memoryless vs. rekurrent, mit vs. ohne LD) sind kontrollierte Vergleiche möglich, ohne zentrale Parameter der Umgebung oder Belohnungsstruktur zu verändern.

Gleichzeitig sind typische Herausforderungen aufgetreten: Die Komplexität der Umgebung führt zu langsamen Lernfortschritten, insbesondere bei Varianten ohne Gedächtnis. Die sorgfältige Wahl initialer Zustände und das Logging zusätzlicher Metriken (Heatmaps, Spielfortschritt) sollen diesem Problem begegnen.

### 4.2 Theoretisch erwartbare Effekte

Basierend auf der Arbeit von Allen et al. [2024] ist anzunehmen, dass LD insbesondere in jenen Spielsituationen Wirkung entfaltet, in denen relevante Informationen über längere Zeiträume verteilt sind. Die rekurrente LD-Variante (v4) könnte hier effizientere interne Repräsentationen aufbauen und zu stabileren Policies führen – ein Effekt, der sich insbesondere im frühen Trainingsverlauf zeigen dürfte. Auch zwischen den Varianten v1 (Frame-Stacking) und v2 (Single-Frame) werden Unterschiede erwartet, da v1 implizit temporale Information nutzt, v2 jedoch nicht.

### 4.3 Ausblick

Die Trainingsläufe aller vier Varianten sollen nach der Abgabe abgeschlossen und systematisch analysiert werden. Geplant ist eine mehrdimensionale Auswertung über klassische RL-Kriterien (Reward, Value Loss), spielinterne Indikatoren (z.B. Arenaleiter, Team-Level) sowie exploratives Verhalten (Heatmaps).

Langfristig bietet das Setup die Möglichkeit, weitere Metriken oder Module zu integrieren – etwa Belief-State-Repräsentationen im Stil von Wang et al. [2023]. Auch eine Erweiterung der Auxiliary-Learning-Komponenten, z.B. durch kombinierte Zielgrößen oder explizite Unsicherheitsabschätzungen, wäre denkbar. Damit könnte LD nicht nur als Diagnoseinstrument für Partial Observability dienen, sondern aktiv zur Verbesserung der Gedächtnisbildung beitragen.

## A Appendix

### A.1 Markov Decision Processes

Ein *Markov Decision Process* (MDP) ist ein mathematisches Modell zur Beschreibung sequenzieller Entscheidungsprobleme mit stochastischer Dynamik. Ein endlicher MDP lässt sich als Tupel  $(S, A, P, R, \gamma)$  beschreiben Sutton and Barto [2018], Murphy [2024], wobei:

- $S$  die endliche Menge aller Zustände ist,
- $A$  die Menge der verfügbaren Aktionen,
- $P(s' | s, a)$  die Übergangswahrscheinlichkeit angibt, vom Zustand  $s$  bei Ausführung der Aktion  $a$  in Zustand  $s'$  zu gelangen,
- $R(s, a) \in \mathbb{R}$  der zugehörige Erwartungswert des Rewards ist, den der Agent für die Aktion  $a$  im Zustand  $s$  erhält,
- $\gamma \in [0, 1]$  der Diskontfaktor ist, der zukünftige Belohnungen abwertet.

Eine *Policy*  $\pi : S \rightarrow \Delta(A)$  ordnet jedem Zustand eine Wahrscheinlichkeitsverteilung über Aktionen zu. Ziel ist es, eine Policy  $\pi$  zu finden, die den erwarteten kumulierten Reward (Return) maximiert:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k}).$$

Die *State-Value Function*  $V^\pi(s)$  beschreibt den erwarteten Return bei Start in Zustand  $s$  und Beachtung der Policy  $\pi$ :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s].$$

Die *Bellman-Gleichung* stellt einen Rekursionszusammenhang für  $V^\pi(s)$  her Sutton and Barto [2018]:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[ R(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) V^\pi(s') \right].$$

Alternativ lässt sich ein MDP auch durch eine partiell definierte Übergangsfunktion  $P : S \times A \rightarrow \mathcal{D}(S)$  beschreiben, wobei  $\mathcal{D}(S)$  die Wahrscheinlichkeitsverteilungen über  $S$  bezeichnet und  $P(s, a, s') := P(s, a)(s')$  gilt Murphy [2024]. Eine Aktion  $a$  ist im Zustand  $s$  erlaubt, wenn  $P(s, a)$  definiert ist.

Eine Policy  $\pi : \text{Paths} \rightarrow \mathcal{D}(A)$  kann dabei entweder stationär (d. h. zustandsbasiert) oder history-basiert sein. Der Spezialfall eines MDPs mit nur einer Aktion pro Zustand entspricht einer *discrete-time Markov chain (DTMC)*.

## A.2 Partially Observable MDPs (POMDPs)

Partially Observable Markov Decision Processes (POMDPs) erweitern das klassische MDP-Framework um die Annahme, dass der Agent den zugrunde liegenden Zustand des Systems nicht direkt beobachten kann. Stattdessen erhält er bei jedem Schritt eine Beobachtung  $o \in \Omega$ , die über eine Beobachtungsfunktion  $\Phi(o \mid s', a)$  mit dem aktuellen Zustand  $s'$  und der vorherigen Aktion  $a$  verknüpft ist.

### Motivation: Warum POMDPs?

In vielen realen Szenarien – von Robotik über medizinische Diagnostik bis hin zu klassischen Spielen wie *Pokémon Red* – hat ein Agent keinen vollständigen Zugriff auf den aktuellen Zustand der Umgebung. In solchen Fällen liefert eine POMDP-Formulierung eine realistischere Modellierung und ermöglicht Lernmethoden, die mit Unsicherheit umgehen können.

### Beobachtungsfunktion $\Phi$ und Beobachtungsraum $\Omega$

Formal definiert sich ein POMDP durch die Erweiterung eines MDPs  $(S, A, P, R)$  um einen endlichen Beobachtungsraum  $\Omega$  und eine Beobachtungsfunktion:

$$\Phi : S \times A \rightarrow \mathcal{D}(\Omega)$$

Die Funktion beschreibt, mit welcher Wahrscheinlichkeit eine bestimmte Beobachtung  $o$  bei gegebener Aktion  $a$  und Zielzustand  $s'$  beobachtet wird.

### Warum Gedächtnis nötig ist

Da der Agent nicht direkt weiß, in welchem Zustand er sich befindet, muss er aus der Folge vergangener Beobachtungen und Aktionen eine sogenannte *Belief State* ableiten – eine Wahrscheinlichkeitsverteilung über mögliche Zustände. Dies erfordert ein internes Gedächtnis bzw. Repräsentationen mit rekursiver Struktur (z. B. RNNs oder LSTMs), um langfristige Abhängigkeiten abzubilden.

### Effektives MDP: Konzept & Limitation

Ein gängiger Ansatz ist es, ein *effektives MDP* zu konstruieren, in dem belief states als neue Zustände verwendet werden. Obwohl dies die Anwendung klassischer MDP-Lösungsmethoden erlaubt, führt es oft zu einem exponentiellen Anstieg der Zustandsdimension (sog. *Belief Space Curse of Dimensionality*) und bleibt damit nur eingeschränkt praktisch anwendbar.

## Markovianität in Repräsentationen

Ein Kernkriterium für erfolgreiche POMDP-Verarbeitung ist die Frage, ob eine Repräsentation existiert, die den Zustand “Markovian” macht – d. h. so informativ, dass zukünftige Entscheidungen optimal allein auf Basis der aktuellen Repräsentation getroffen werden können. Dieser Aspekt steht im Zentrum moderner Ansätze zur partiellen Beobachtbarkeit – etwa der *Lambda Discrepancy*, die misst, wie stark aktuelle Repräsentationen von der Markov-Eigenschaft abweichen.

## A.3 TD( $\lambda$ ) und $\lambda$ -Returns

Temporal-Difference-Learning mit  $\lambda$  verbindet die Vorteile von Monte-Carlo-Methoden und  $n$ -step-TD-Updates durch eine gewichtete Interpolation aller möglichen Returns Sutton [1988], Sutton and Barto [2018]. Es stellt damit ein zentrales Verfahren zur Wertabschätzung bei sequentiellen Entscheidungsprozessen dar.

### Interpolierte Returns

Statt nur eine feste Anzahl von Zeitschritten für die Schätzung zu berücksichtigen (wie bei  $n$ -step TD), nutzt TD( $\lambda$ ) eine geometrisch gewichtete Summe aller möglichen  $n$ -step>Returns [Sutton and Barto, 2018, Kap.12]:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

Hierbei ist  $G_t^{(n)}$  der  $n$ -step-Return zum Zeitpunkt  $t$ , und  $\lambda \in [0, 1]$  ein Hyperparameter.

### Zustandswertfunktion

Mit diesem  $\lambda$ -Rückgabewert kann eine Schätzung der Zustandswertfunktion  $V(s)$  erfolgen, typischerweise per Online-Update:

$$V(s_t) \leftarrow V(s_t) + \alpha \cdot \delta_t^\lambda$$

wobei  $\delta_t^\lambda = G_t^\lambda - V(s_t)$  den TD-Fehler darstellt ?.

### Spezialfälle

- $\lambda = 0$ : entspricht TD(0) – nur 1-step-Return wird genutzt.
- $\lambda = 1$ : entspricht Monte Carlo – nur vollständiger Episodenreturn zählt.

### Konvergenzeigenschaften

Für tabellarische Umgebungen und geeignete Lernraten konvergiert TD( $\lambda$ ) unter gewissen Bedingungen zur korrekten Erwartung der Belohnung [Sutton and Barto, 2018, Kap.12]. Besonders vorteilhaft ist TD( $\lambda$ ), wenn der Lernprozess inkrementell und kontinuierlich erfolgen soll.

### Relevanz bei POMDPs

Bei nicht-markovianischen Zustandsrepräsentationen weichen Rückgaben mit unterschiedlichen  $n$ -Schritten stark voneinander ab. TD( $\lambda$ ) kann daher – über die entstehenden Diskrepanzen – implizit auf die „Qualität“ der Repräsentation hinweisen. Genau dieses Prinzip nutzt die Lambda Discrepancy als Maß [Allen et al., 2024, Abschnitte2.3–3.1].

## A.4 Lambda Discrepancy: Formale Grundlage

Die *Lambda Discrepancy* ( $\Lambda_{\lambda_1, \lambda_2}$ ) ist ein Maß dafür, wie stark sich die Wertfunktionen  $Q^\lambda$  unterscheiden, wenn man verschiedene Rückgabewichtungen  $\lambda \in [0, 1]$  verwendet. Diese Abweichung wird insbesondere in POMDPs als Indikator für mangelnde Markovianität der Beobachtungsrepräsentationen genutzt Allen et al. [2024].



### Definition

Seien  $\lambda_1, \lambda_2 \in [0, 1]$ . Die *Lambda Discrepancy* ist definiert als:

$$\Lambda_{\lambda_1, \lambda_2} = \|Q^{\lambda_1} - Q^{\lambda_2}\|_{\mu}$$

wobei  $\|\cdot\|_{\mu}$  eine gewichtete Norm über die Zustandsverteilung  $\mu$  ist (z. B. L2-Norm).  $Q^{\lambda}$  bezeichnet den erwarteten  $\lambda$ -Return einer bestimmten Repräsentation im Beobachtungsraum.

### Hintergrund und Bedeutung der Kennzahl

In vollständig beobachtbaren MDPs stimmen alle  $Q^{\lambda}$  überein, da die Wahl des Rückgabewichtungsparameters keinen Einfluss auf den optimalen Erwartungswert hat. In POMDPs hingegen kann die Wahl von  $\lambda$  zu systematischen Abweichungen führen – ein Hinweis darauf, dass die Beobachtungsrepräsentation nicht alle relevanten Informationen enthält [Allen et al., 2024, Abschnitt 2.3].

### Theoretische Aussage

Ein grundlegender Befund aus Allen et al. [2024] lautet:

**Satz:**  $\Lambda_{\lambda_1, \lambda_2} = 0$  für alle  $\lambda_1, \lambda_2 \in [0, 1]$  genau dann, wenn die zugrunde liegende Repräsentation den Prozess *Markovian* macht (z. B. in einem Block-MDP).

Diese Eigenschaft ermöglicht es, die *Markovianität* einer Repräsentation zu beurteilen – auch ohne direkten Zugriff auf die latenten Zustände.

### Praktische Einbindung als auxiliary loss

In der Praxis wird die Lambda Discrepancy als zusätzlicher Verlustterm  $\mathcal{L}_{LD}$  zur PPO-Zielfunktion hinzugefügt. Die kombinierte Verlustfunktion lautet Allen et al. [2024]:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}} + \alpha \cdot \mathcal{L}_{LD}$$

Dabei bezeichnet  $\alpha \in \mathbb{R}_{\geq 0}$  einen Hyperparameter, der den relativen Einfluss der Lambda Discrepancy gegenüber dem Hauptziel (Policy-Optimierung via PPO) reguliert. Ein zu hoher Wert kann das Haupttraining überlagern, ein zu niedriger Wert hingegen den Effekt des auxiliary loss unterdrücken.

In Analogie zu anderen auxiliary learning approaches (etwa value-prediction, curiosity oder contrastive learning) fungiert  $\alpha$  als Balancing-Faktor und muss meist empirisch bestimmt werden.

## A.5 Gedächtnislernen bei POMDPs

Partiell beobachtbare Umgebungen erfordern vom Agenten, relevante Informationen aus der Vergangenheit zu speichern. Die Frage, *was* und *wie* erinnert werden soll, steht im Zentrum moderner Gedächtnisarchitekturen im Reinforcement Learning.

### Speicherfunktionen als rekursive Operatoren

Ein geläufiges Konzept zur Beschreibung von Gedächtnisprozessen ist die **Memory Function**  $\mu$ , wie sie etwa von Allen et al. [2024] eingeführt wird:

$$m_t = \mu(\omega_t, a_{t-1}, m_{t-1}),$$

wobei  $\omega_t$  die aktuelle Beobachtung,  $a_{t-1}$  die vorherige Aktion und  $m_{t-1}$  der vorherige Gedächtniszustand ist. Der Agent lernt eine solche Funktion implizit, wenn er mit rekurrenten Modellen wie LSTMs oder GRUs ausgestattet ist.

## Gedächtnisbasierte Repräsentationen in POMDPs

Durch Erweiterung der Beobachtungsfunktion  $\Phi$  zu einer *Historienfunktion*  $\Phi^*$ , die den Zustand des Gedächtnisses berücksichtigt, entsteht eine Repräsentation, die die POMDP in ein effektives MDP überführt. Ziel ist es, eine Darstellung zu erlernen, die ausreicht, um optimale Entscheidungen allein auf Basis des Gedächtniszustands zu treffen – vgl. Allen et al. [2024].

### Lernziel: Minimierung der Lambda Discrepancy

Die Lambda Discrepancy  $\Lambda_{\lambda_1, \lambda_2}$  kann dabei als **Trainingssignal** fungieren, das signalisiert, wie gut das Gedächtnis die Markov-Eigenschaft herstellt:

$$\Lambda_{\lambda_1, \lambda_2} = \|Q^{\lambda_1} - Q^{\lambda_2}\|.$$

Der Gradient dieses Ausdrucks fließt durch die Gedächtnisfunktion  $\mu$ , wodurch der Agent explizit Repräsentationen optimiert, die kausal relevante Informationen akkumulieren Allen et al. [2024].

### Empirische Architektur: LSTM als Speicher

In der Praxis wird  $\mu$  häufig durch rekurrente neuronale Netzwerke realisiert – insbesondere LSTMs, die Informationen durch spezialisierte Speicher- und Vergessensmechanismen über längere Zeiträume hinweg halten können. Eine der frühesten erfolgreichen Anwendungen im RL-Kontext wurde von Hausknecht and Stone [2015] gezeigt, die ein rekurrentes Deep-Q-Network (DRQN) für POMDPs vorschlugen.

## References

- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.
- C. Allen, A. Kirtland, R. Y. Tao, S. Lobel, and D. Scott. Mitigating partial observability in sequential decision processes via the lambda discrepancy. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2407.07333>.
- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Andrew Wang, Andrew C. Li, Bo Dai, Alexei Baevski, and Aditya Grover. Learning belief representations for partially observable deep rl. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, pages 34779–34797. PMLR, 2023. URL <https://proceedings.mlr.press/v202/wang23p.html>.
- Elizabeth Gilmour, Noah Plotkin, and Leslie N. Smith. An approach to partial observability in games: Learning to both act and observe. *IEEE Conference on Games*, 2021. doi:10.1109/cog52621.2021.9619004. URL <https://doi.org/10.1109/cog52621.2021.9619004>.
- Lingheng Meng, Rob Gorbet, and Dana Kulić. Memory-based deep reinforcement learning for pomdps. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5619–5626, Prague, Czechia, 2021. IEEE. doi:10.1109/IROS51168.2021.9636140. URL <https://ieeexplore.ieee.org/document/9636140/>.
- Jacob N. Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016. URL [https://proceedings.neurips.cc/paper\\_files/paper/2016/file/55b1927f2c6662b2d9b3f5580b4f249e-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2016/file/55b1927f2c6662b2d9b3f5580b4f249e-Paper.pdf).
- Kevin P. Murphy. Reinforcement learning: An overview. *arXiv preprint arXiv:2412.05265*, 2024. URL <https://arxiv.org/abs/2412.05265>. Kapitel 1.2.1–1.2.2 besonders relevant für MDP/POMDP.
- Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi:10.1007/BF00115009. URL <https://doi.org/10.1007/BF00115009>.
- Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *Proceedings of the AAAI Fall Symposium Series*, 2015. URL <https://arxiv.org/abs/1507.06527>.